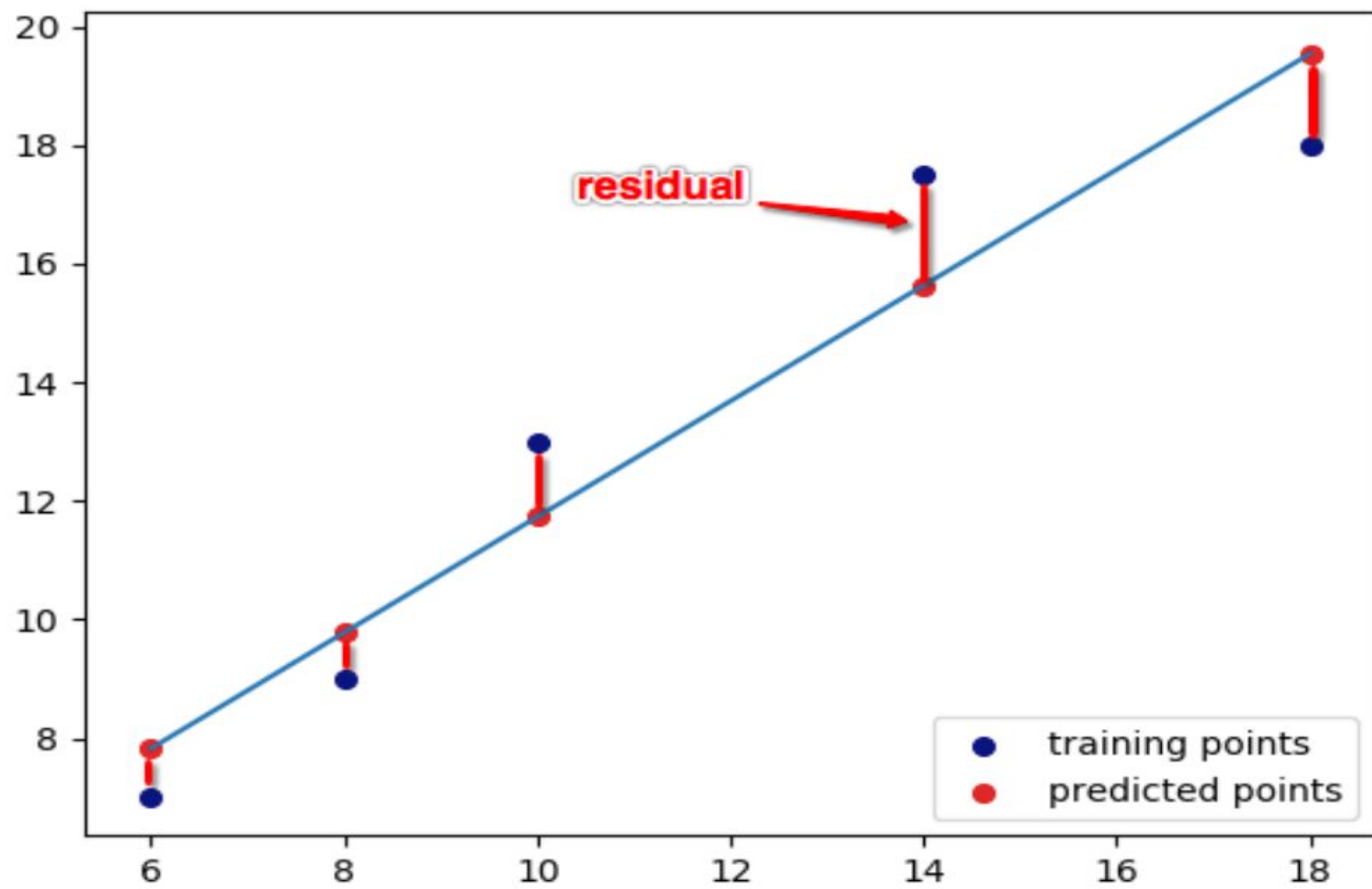


Polynomial Regression



- Polynomial regression is a type of regression analysis where the relationship between the independent variable (usually denoted as x) and the dependent variable (usually denoted as y) is modeled as an **n th-degree polynomial function**.
- **The general form of an n th-degree polynomial is:**
- $$f(x) = \beta_0 + \beta_1 x^1 + \beta_2 x^2 + \beta_3 x^3 + \dots + \beta_n x^n$$
- Here, n is the degree of the polynomial, and $\beta_0, \beta_1, \beta_2, \dots, \beta_n$ are the coefficients that are determined during the model training process.

- In the context of regression, the goal is to find the best-fitting polynomial that describes the relationship between x and the conditional mean of y .
- $E(y | x)$, which represents the expected value of y given a specific value of x . So, the polynomial regression model can be expressed as:

$$E(y | x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \dots + \beta_n x^n$$

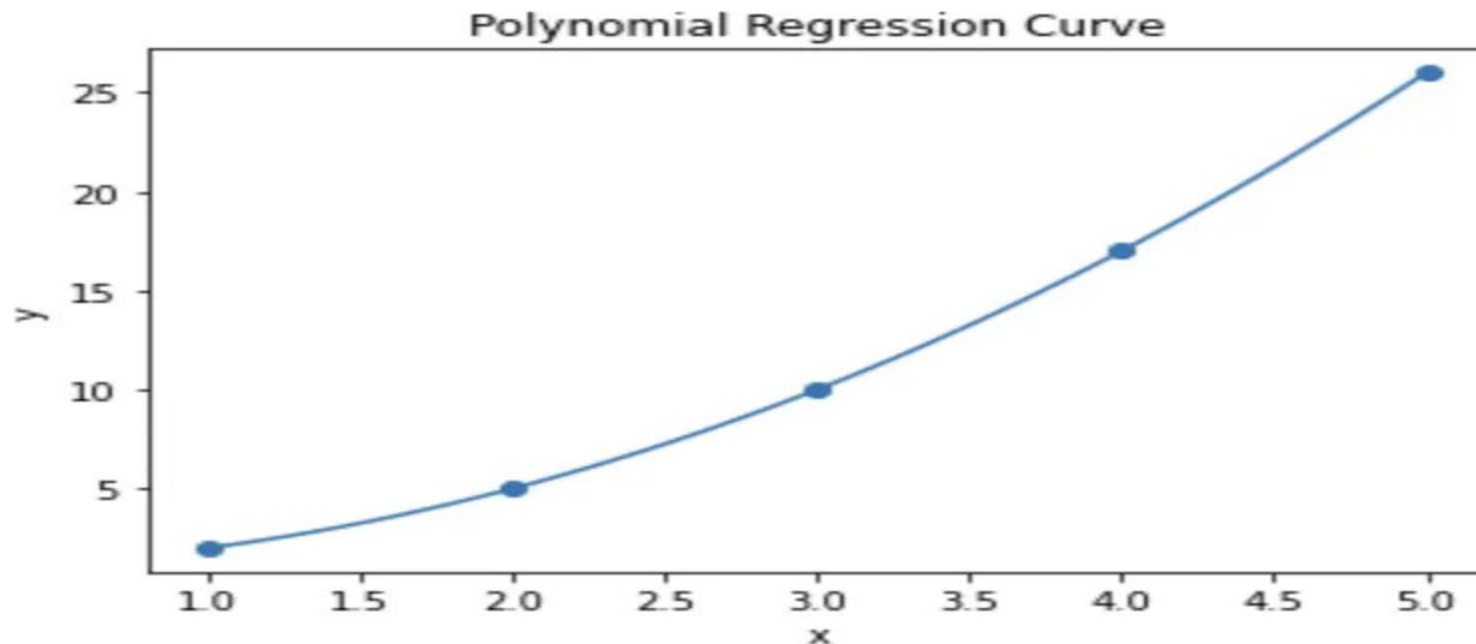
- The idea is to choose the values of the coefficients ($\beta_0, \beta_1, \beta_2, \dots, \beta_n$) in such a way that the predicted values of y based on the polynomial equation are as close as possible to the actual observed values of y in the dataset.

Polynomial regression is useful when the relationship between the variables is not linear and can be better approximated by a polynomial curve. However, it's important to be cautious about overfitting, especially when using higher-degree polynomials, as they can lead to models that fit the training data too closely and may not generalize well to new data. Regularization techniques can be applied to mitigate overfitting in polynomial regression models.

Types

- Linear, when the degree is 1.
- Quadratic, the degree of this equation is 2.
- Cubic with a degree as three continues, based on the degree used.

- The equation for Polynomial regression is:
- # Equation for polynomial regression can be represented as:
- $y = b_0 + b_1x + b_2x^2 + b_3x^3 + \dots + b_nx^n$



- Polynomial regression defines the non-linear phenomena, such as:
- Progression of disease epidemics.
- The growth rates of several tissues.
- The distribution of carbon isotopes in the lake sediments.

- # Step 1: Importing libraries and datasets
- import numpy as np
- import pandas as pd
- import matplotlib.pyplot as plt
- from sklearn.model_selection import train_test_split
- from sklearn.linear_model import LinearRegression
- from sklearn.preprocessing import PolynomialFeatures
- from sklearn.metrics import mean_squared_error

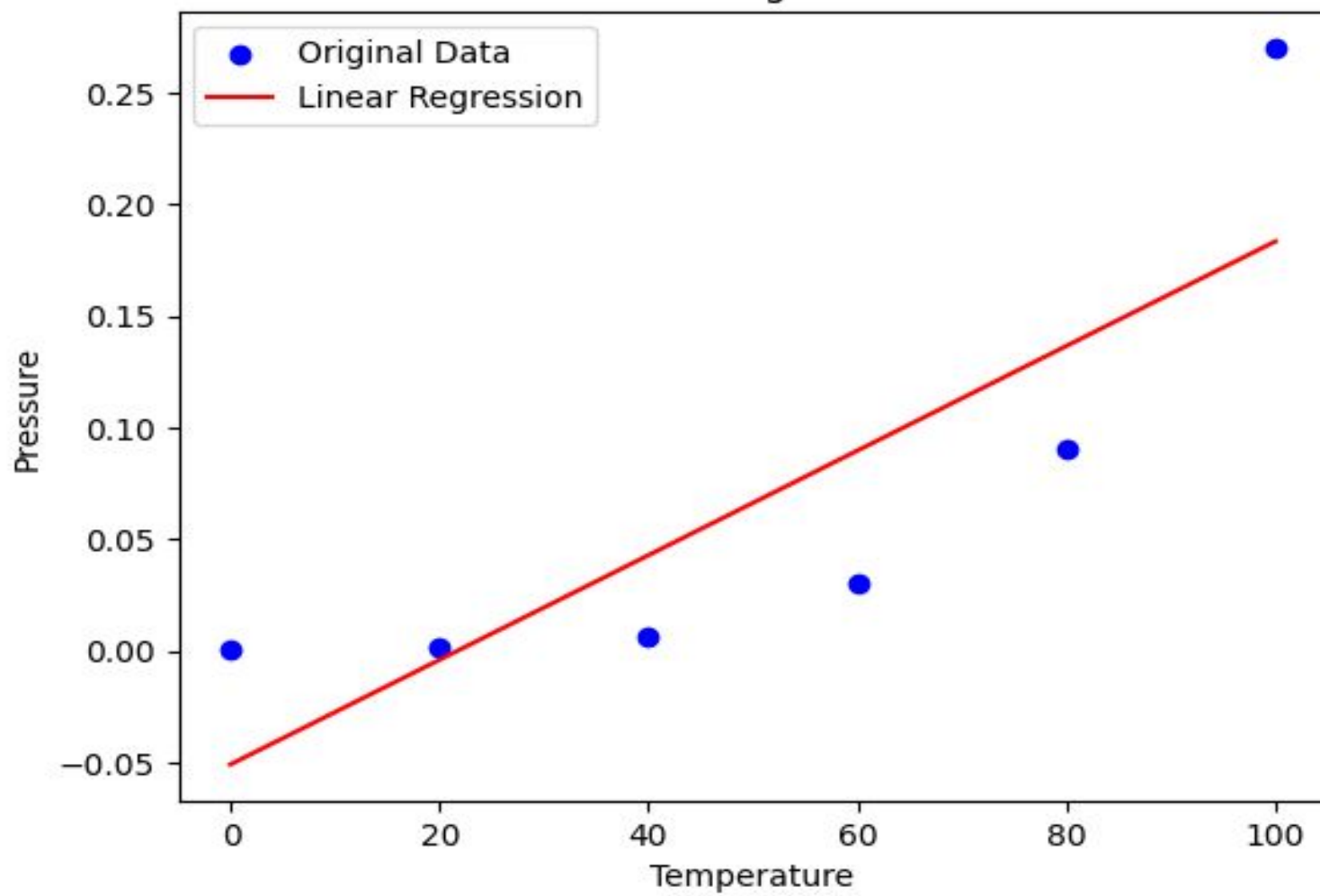
- # Assuming you have a dataset named 'data' with two columns 'X1' and 'X2', and 'Y' as the target variable.
- data = pd.read_csv("/content/data.csv")
-
- data=data.dropna()
- # Divide dataset into two components, X and y. X will contain the Column between 1 and 2. y will contain the 2nd column.
- X = data.iloc[:, 1:2].values
- y = data.iloc[:, 2].values

```
# Step 3: Fitting Linear Regression to the dataset
# Fitting Linear Regression to the dataset
from sklearn.linear_model import LinearRegression
lin = LinearRegression()
lin.fit(X, y)
```

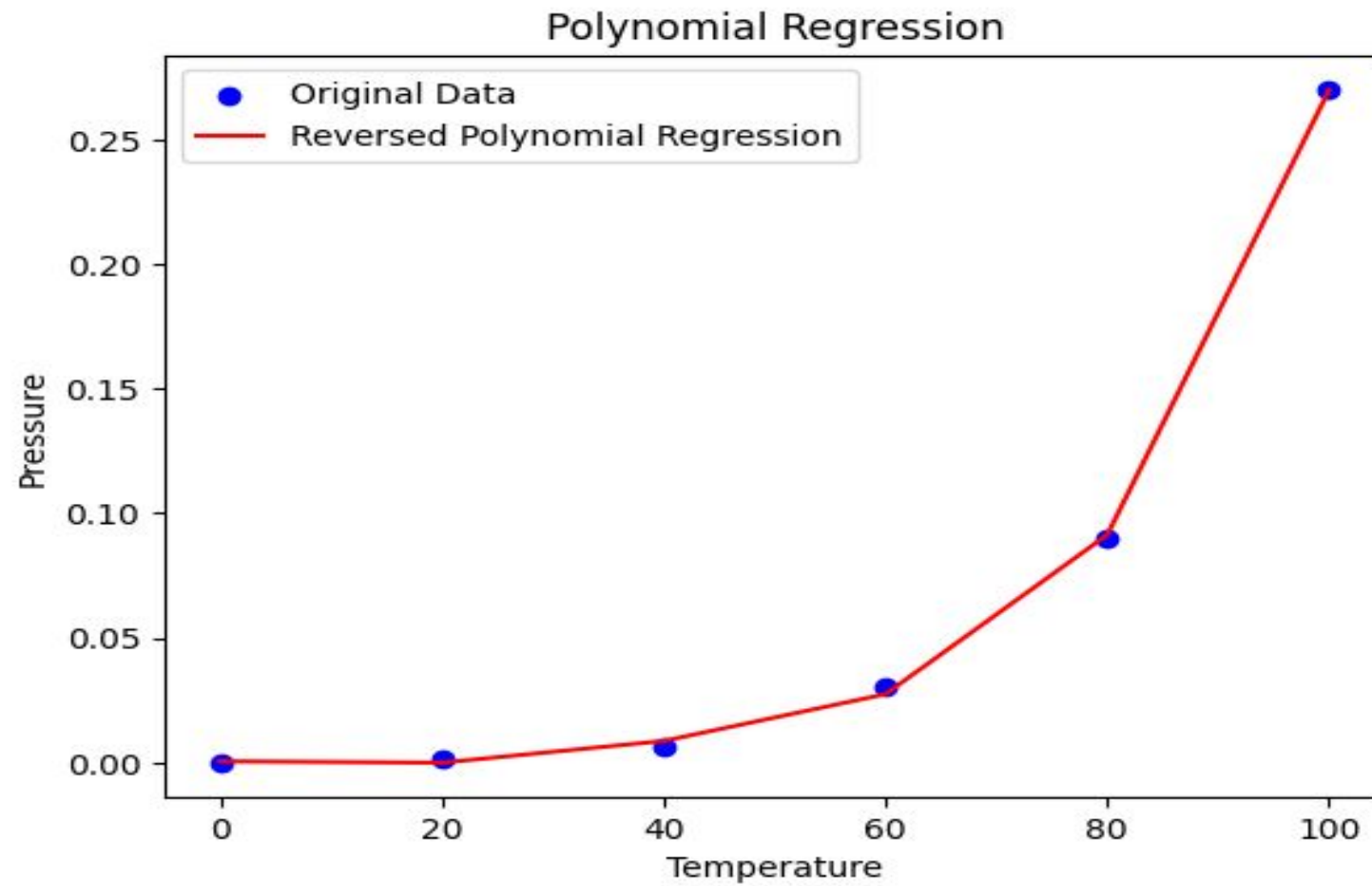
- # Step 4: Fitting Polynomial Regression to the dataset
- # Fitting Polynomial Regression to the dataset
- poly_degree = 4
- poly =
PolynomialFeatures(degree=poly_degree)
- X_poly = poly.fit_transform(X)
- #poly.fit(X_poly, y)
- lin2 = LinearRegression()
- lin2.fit(X_poly, y)

- # Step 5: Visualizing the Linear Regression results using a scatter plot.
- # Visualising the Linear Regression results
- `plt.scatter(X, y, color='blue', label='Original Data')`
- `plt.plot(X, lin.predict(X), color='red', label='Linear Regression')`
- `plt.title('Linear Regression')`
- `plt.xlabel('Temperature')`
- `plt.ylabel('Pressure')`
- `plt.legend()`
- `plt.show()`

Linear Regression



- # Step 6: Visualizing the Polynomial Regression results using a scatter plot.
- # Visualising the Polynomial Regression results
- `plt.scatter(X, y, color='blue', label='Original Data')`
- `plt.plot(X, lin2.predict(poly.fit_transform(X)), color='red', label='Reversed Polynomial Regression')`
- `plt.title('Polynomial Regression')`
- `plt.xlabel('Temperature')`
- `plt.ylabel('Pressure')`
- `plt.legend()`
- `plt.show()`



Linear Regression Prediction: [0.20675333]

Polynomial Regression Prediction: [0.43295877]

- `# Predicting a new result with Linear Regression`
- `new_data_point_linear = np.array([[110.0]])`
- `prediction_linear =
lin.predict(new_data_point_linear)`
- `print(f'Linear Regression Prediction:
{prediction_linear}')`

- # Predicting a new result with Polynomial Regression
- new_data_point_poly = np.array([[110.0]])
- prediction_poly =
lin2.predict(poly.fit_transform(new_data_point_poly))
- print(f'Polynomial Regression Prediction:
{prediction_poly}')

Logistic regression

- Logistic regression is a supervised machine learning algorithm mainly used for classification tasks where the goal is to predict the probability that an instance of belonging to a given class or not.
- It is a kind of statistical algorithm, which analyze the relationship between a set of independent variables and the dependent binary variables.
- It is a powerful tool for decision-making. For example email spam or not.

- It is used for predicting the categorical dependent variable using a given set of independent variables.
- Logistic regression predicts the output of a categorical dependent variable. Therefore the outcome must be a categorical or discrete value.
- It can be either Yes or No, 0 or 1, true or False, etc. but instead of giving the exact value as 0 and 1, it gives the probabilistic values which lie between 0 and 1.

- The main mechanics behind logistic regression as a probabilistic model for binary classification, let's first introduce the odds: the odds in favor of a particular event. The odds can be written as

$$\frac{p}{(1-p)}$$

- where p stands for the probability of the positive event. The term “positive event” does not necessarily mean “good,” but refers to the event that we want to predict, for example, the probability that a patient has a certain disease given certain symptoms; we can think of the positive event as class label $y = 1$ and the symptoms as features x .
- Hence, for brevity, we can define the probability p as $p := p(y = 1|x)$, the conditional probability that a particular example belongs to a certain class 1 given its features, x .

- We can then further define the logit function, which is simply the logarithm of the odds (log-odds):

$$\text{logit}(p) = \log \frac{p}{(1-p)}$$

The logit function takes input values in the range 0 to 1 and transforms them into values over the entire real-number range.

Under the logistic model, we assume that there is a linear relationship between the weighted inputs and the log-odds:

$$\text{logit}(p) = w_1x_1 + \cdots + w_mx_m + b = \sum_{i=1}^m w_ix_i + b = \mathbf{w}^T \mathbf{x} + b$$

This inverse of the logit function is typically called the **logistic sigmoid function**, which is sometimes simply abbreviated to **sigmoid function** due to its characteristic S-shape:

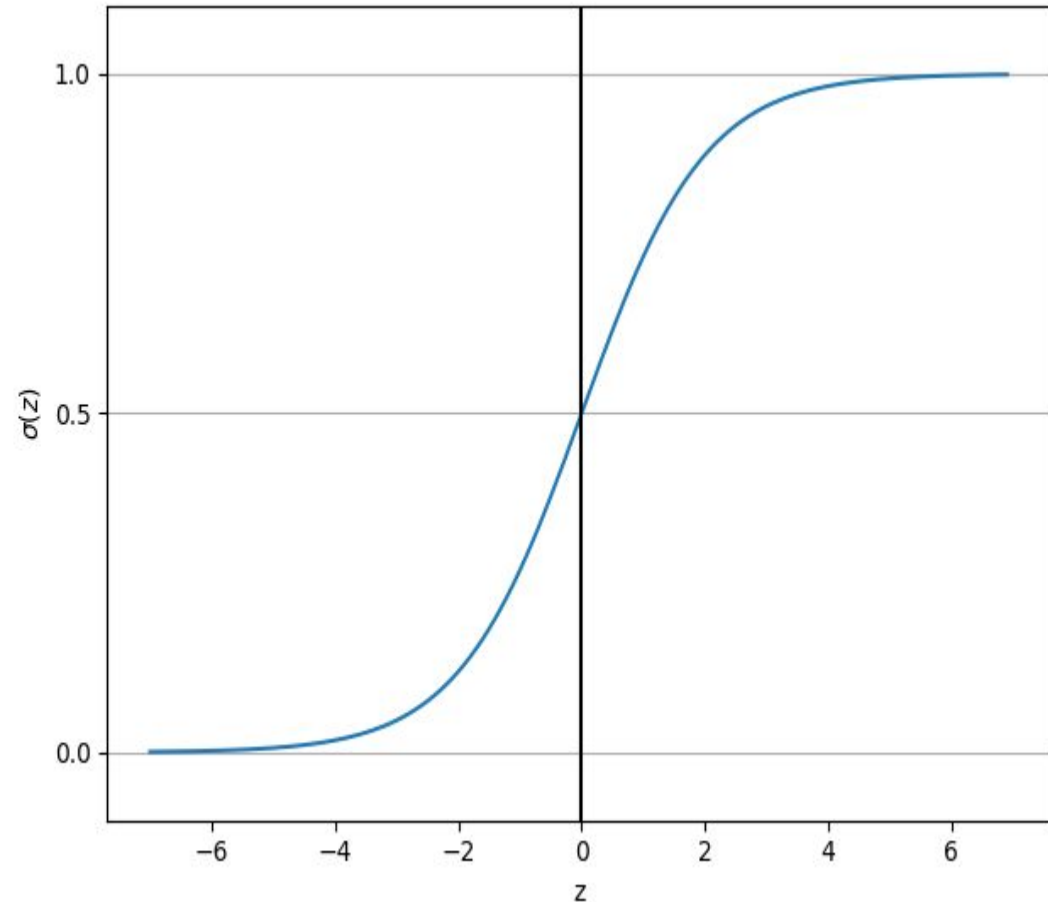
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Here, z is the net input, the linear combination of weights, and the inputs (that is, the features associated with the training examples):

$$z = \mathbf{w}^T \mathbf{x} + b$$

Now, let's simply plot the sigmoid function for some values in the range -7 to 7 to see how it looks:

```
import matplotlib.pyplot as plt
import numpy as np
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))
z = np.arange(-7, 7, 0.1)
sigma_z = sigmoid(z)
plt.plot(z, sigma_z)
plt.axvline(0.0, color='k')
plt.ylim(-0.1, 1.1)
plt.xlabel('z')
plt.ylabel('$\sigma(z)$')
# y axis ticks and gridline
plt.yticks([0.0, 0.5, 1.0])
ax = plt.gca()
ax.yaxis.grid(True)
plt.tight_layout()
plt.show()
```



- We can see that $\sigma\sigma(z)$ approaches 1 if z goes toward infinity ($z \rightarrow \infty$) since e^{-z} becomes very small for large values of z . Similarly, $\sigma\sigma(z)$ goes toward 0 for $z \rightarrow -\infty$ as a result of an increasingly large denominator.
- Thus, we can conclude that this sigmoid function takes real-number values as input and transforms them into values in the range $[0, 1]$ with an intercept at $\sigma\sigma(0) = 0.5$.

The predicted probability can then simply be converted into a binary outcome via a threshold function:

$$\hat{y} = \begin{cases} 1 & \text{if } \sigma(z) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

If we look at the preceding plot of the sigmoid function, this is equivalent to the following:

$$\hat{y} = \begin{cases} 1 & \text{if } z \geq 0.0 \\ 0 & \text{otherwise} \end{cases}$$

How does Logistic Regression work?

- The logistic regression model transforms the linear regression function continuous value output into categorical value output using a **sigmoid function**, which maps any real-valued set of independent variables input into **a value between 0 and 1**.
- This function is known as the **logistic function**.

Let the independent input features be

$$X = \begin{bmatrix} x_{11} & \dots & x_{1m} \\ x_{21} & \dots & x_{2m} \\ \vdots & \ddots & \vdots \\ x_{n1} & \dots & x_{nm} \end{bmatrix}$$

and the dependent variable is Y having only binary value i.e. 0 or 1.

$$Y = \begin{cases} 0 & \text{if } \textit{Class 1} \\ 1 & \text{if } \textit{Class 2} \end{cases}$$

Then apply the multi-linear function to the input variables X

$$z = (\sum_{i=1}^n w_i x_i) + b$$

Here x_i is the i th observation of X , $w_i = [w_1, w_2, w_3, \dots, w_m]$ is the weights or Coefficient, and b is the bias term also known as intercept. simply this can be represented as the dot product of weight and bias.

$$z = w \cdot X + b$$

Logistic Regression Equation

- The odd is the ratio of something occurring to something not occurring.
- it is different from probability as the probability is the ratio of something occurring to everything that could possibly occur.

so odd will be $\frac{p(x)}{1-p(x)} = e^z$

Applying natural log on odd. then log odd will be

$$\log \left[\frac{p(x)}{1-p(x)} \right] = z$$

$$\log \left[\frac{p(x)}{1-p(x)} \right] = w \cdot X + b$$

Type equation here.

Applying exponential function on both sides

$$\frac{p(x)}{1-p(x)} = e^{w \cdot X + b}$$

$$p(x) = (1 - p(x)) \times (e^{w.X+b})$$

$$p(x) = e^{w.X+b} - p(x).e^{w.X+b}$$

$$p(x) + p(x).e^{w.X+b} = e^{w.X+b}$$

$$p(x)(1 + e^{w.X+b}) = e^{w.X+b}$$

$$p(x) = \frac{e^{w.X+b}}{1 + e^{w.X+b}}$$

Then the final logistic regression equation will be:

$$p(X; b, w) = \frac{e^{w.X+b}}{1 + e^{w.X+b}}$$

- Now dividing by $e^{w.X+b}$, we will get:

$$p(X; b, w) = \frac{1}{1 + e^{-(w.X+b)}}$$

